

✅ 문제 해결과정 → 비즈니스 로직에 산재되어 있는 암호화 처리 로직을 AOP와 애너테이션을 활용하여 적절히 분리함



내용 요약

- 현재 문제는 회원 서비스 오브젝트에 비즈니스 로직과 기술적 로직이 혼재되어 있습니다.
- 또한, 해당 기술적 로직은 여러 부분에 나타나고 있습니다.
- 이는 서비스 오브젝트에 작성된 비즈니스 로직에 대한 가독성을 저하시킬 뿐만 아니라, 변경에도 유연하게 대처할 수 없습니다.
- 이 문제를 해결하고자 **AOP와 애너테이션을 활용하여 비즈니스 로직과 기술적 로직을 분리했습니다.**

!/? 기존 프로젝트에서의 문제점

▼ 문제 코드

```
@Service
@Transactional
public class MemberServiceImpl implements UserDetailsService {
    private final MemberConnectionHistoryRepository memberConnectionHistoryRepository;
    private final MemberRepository memberRepository;
    private final MemberStateRepository memberStateRepository;
    // 생략

    public Member create(MemberRegisterRequest request) {
        try {
            // 중복되는 이메일 확인
```

```

        checkDuplicatedEmail(request.getEmail());
        // 중복되는 아이디가 있는지 확인
        checkDuplicatedId(request.getId());
        // 회원 약관 유효성 확인
        request.checkValidTerms();
        request.checkValidTermsCount();
    } catch (MemberDuplicatedEmailException
            | MemberDuplicatedIdException
            | MemberMandatoryTermNotAgreedException
            | MemberWrongCountTermCondition e) {
        // 회원가입 처리가 불가능할 경우 컨트롤러에 비즈니스 예외 전달
        throw e;
    }

    // 회원가입 처리, 서버 내부 예외 발생시 재시도를 통한 복구 작업 진행
    return retryableCreateMember(request);
}

@Transactional
@Retryable(
    value = {RuntimeException.class},
    maxAttempts = MAX_RETRY,
    backoff = @Backoff(delay = RETRY_DELAY))
public Member retryableCreateMember(MemberRegisterRequest request) {
    encodePassword(request);
    Member savedMember = saveMember(request);
    saveMemberTermsHistory(request, savedMember);
    saveMemberState(savedMember, "MBS001"); // 초기 상태 코드 : MBS0
    return savedMember;
}

@Recover
public Member recover(RuntimeException e) {
    throw new RetryFailedException();
}

private void encodePassword(MemberRegisterRequest request) {
    String encodedPassword = passwordEncoder.encode(request.getPas

```

```

        request.setPassword(encodedPassword);
    }

}

```

(프로젝트 url : https://github.com/jongheonleee/FinalProject_5RE5_BE)



코드 설명

- 기능 : 회원가입 처리
 - 유효성 검증
 - 중복되는 이메일 확인
 - 중복되는 아이디 확인
 - 필수 약관 동의 여부 확인
 - 회원가입 처리
 - **비밀번호 암호화 ← 문제 지점**
 - 회원 등록
 - 회원 약관 이력 등록
 - 회원 초기 상태 등록
 - 회원가입 등록 실패 시 재시도를 통한 복구 처리
 - 재시도 정책에 따라 재시도 처리

- 위 코드는 24년도 12월달에 팀 프로젝트에서 회원 파트를 담당할 때 작성한 코드입니다.
- 지금 돌이켜 보았을 때, 많은 문제점들이 보입니다. 하지만, 여기서 얘기하고 싶은 가장 큰 문제점은 **비즈니스 로직에 기술적 로직이 혼재되어 있는 것입니다. 또한, 이 기술적 로직은 다른 부분에서도 나타나고 있습니다. 즉, 기술적 로직이 여러 기능에 산재되어 나타납니다.**
 - **비즈니스 로직에는 업무 프로세스 흐름이 반영되어야합니다.**
 - 따라서, 비즈니스 로직을 담고있는 서비스 오브젝트는 추상화 레벨을 높여서 해당 코드만 봐도 프로세스 흐름을 파악할 수 있어야합니다.

- 하지만, **비밀번호 암호화 처리와 같은 기술적 로직은 업무 프로세스와 무관합니다.**
- 또한, 이런 기술적 로직은 언제든지 변경될 수 있습니다.
- 이를 고려해서 해당 서비스 객체에서 비밀번호 암호화 처리와 같은 기술적 로직을 배치하는 것은 좋지 않다고 생각합니다.
- 따라서, 이러한 문제를 **AOP와 애너테이션을 적절히 활용하여 특정 API 요청에 대해서 비밀번호 암호화 처리가 될 수 있도록 구성했습니다.**

💡 문제 해결 접근법



- 문제 정의: 기술적 로직이 서비스 오브젝트에 산재되어 있다
- 해결 방안: AOP와 애너테이션을 활용하여 비즈니스 로직으로부터 기술적 로직을 분리해내자
 1. 애노테이션 정의
 2. 적용할 필드에 애노테이션 선언
 3. AOP 정의

1. 애노테이션 정의

```
@Target(AnnotationTarget.FIELD)
@Retention(AnnotationRetention.RUNTIME)
annotation class PasswordEncryption
```

- 애노테이션이란?
 - 애노테이션은 메타 데이터입니다.
 - 좀더 면밀하게 얘기하자면, **프로그램의 소스코드 안에 다른 프로그램을 위한 정보를 미리 약속한 형식으로 포함시킨 것이 바로 애너테이션입니다.**
 - 애너테이션은 주석처럼 프로그래밍 언어에 영향을 미치지 않으면서도 다른 프로그램에게 유용한 정보를 제공할 수 있다는 장점이 있습니다.

- 즉, 컴파일러, 개발 도구, 프레임워크 등이 애너테이션을 활용하여 특정 작업을 수행할 수 있음
- 해당 애노테이션은 컴파일러를 위한 메타 데이터입니다.
- 이를 통해 **컴파일러는 어떤 필드 값을 암호화 처리할지 판단할 수 있습니다.**

2. 적용할 필드에 애노테이션 선언

```
data class SignupRequest(
    @field:Email(message = "유효한 이메일 형식이 아닙니다.")
    @field:NotBlank(message = "이메일을 입력해주세요.")
    val email: String,

    @field:NotBlank(message = "이름을 입력해주세요.")
    @field:Size(min = 2, max = 20, message = "이름은 2자 이상 20자 이하로 입력해주세요.")
    val name: String,

    @PasswordEncryption
    @field:NotBlank(message = "비밀번호를 입력해주세요.")
    @field:Size(min = 8, max = 30, message = "비밀번호는 8자 이상 30자 이하로 입력해주세요.")
    var password: String,

    @field:NotBlank(message = "포지션을 선택해주세요.")
    val positionId: String,

    @field:Size(max = 200, message = "자기소개는 최대 200자까지 입력 가능합니다.")
    val introduce: String
) {
    fun toCommand(): UserSignupCommand {
        return UserSignupCommand(
            email = email,
            name = name,
            password = password,
            positionId = positionId,
            introduce = introduce
        )
    }
}
```

```
}
}
```

- password 필드에 **@PasswordEncryption**을 선언하여 컴파일러에게 해당 필드가 암호화 대상임을 알림
 - 여기서, 주의할 점은 해당 필드는 추후에 AOP를 통해 암호화 처리되기 때문에 변수를 선언할 때 var 가변 형식으로 선언해야함

3. AOP 구현

```
@Aspect
@Component
@RequiredArgsConstructor
class PasswordEncryptionAspect(
    private val passwordEncoder: PasswordEncoder
) {
    @Around("execution(* com.backend.sodam.domain.users.controller.*(..)")
    fun passwordEncryptionAspect(pjp: ProceedingJoinPoint): Any {
        Arrays.stream(pjp.args)
            .forEach { fieldEncryption(it) }
        return pjp.proceed()
    }

    private fun fieldEncryption(obj: Any?) {
        if (ObjectUtils.isEmpty(obj)) return

        FieldUtils.getAllFieldsList(obj!!::class.java)
            .filter { !Modifier.isFinal(it.modifiers) && !Modifier.isStatic(it.modifiers) }
            .forEach { field →
                try {
                    if (!field.isAnnotationPresent(PasswordEncryption::class.java)) return

                    val encryptionField = FieldUtils.readField(field, obj, true)

                    if (encryptionField !is String) return@forEach

                    val encryptedPassword = passwordEncoder.encode(encryptionField)
```

```

        FieldUtils.writeField(field, obj, encryptedPassword, true)
    } catch (e: Exception) {
        throw RuntimeException(e)
    }
}
}
}
}

```



코드 설명

- 기능 : AOP 처리 → 특정 요청으로 전달받은 DTO에 비밀번호 필드 암호화 처리
 - 부연 설명
 - 포인트 컷 명시,
com.backend.sodam.domain.users.controller 패키지 이하의 모든 클래스에 있는 모든 메서드가 AOP 적용 대상
 - **ProceedingJoinPoint**는 현재 실행 중인 메서드 호출에 대한 모든 정보를 담고 있으며 해당 메서드를 실행 시키거나 실행 시키지 않게 하는 오브젝트
 - **@Around**는 핵심 조인 포인트가 되는 메서드의 실행 전후 둘 다 관연할 수 있도록 하는 어드바이스
 - .filter { !Modifier.isFinal(it.modifiers) &&
!Modifier.isStatic(it.modifiers)} 이 부분을 통해서 final 또는 static 이 아닌 필드만 필터링
 - 필터링된 필드 중에 **PasswordEncryption** 이 선언된 필드를 찾고 암호화 처리 진행

★ 성과

▼ 결과 코드

@Service

```

@RequiredArgsConstructor
class UserService(
    private val fetchSocialUserPort: FetchSocialUserPort,
    private val fetchUserPorts: List<FetchUserPort>,
    private val createNormalUserPort: CreateNormalUserPort,
    private val createSocialUserPort: CreateSocialUserPort,
    private val updateUserPorts: List<UpdateUserPort>,
    // 생략
) : FetchUserUseCase, RegisterUserUseCase, UpdateUserUseCase, DeleteUserUseCase {

    // 📌 실제 핵심 비즈니스 로직
    @Transactional
    override fun registerNormalUser(userSignupCommand: UserSignupCommand): UserSignupResponse {
        // 회원등록 작업 전 유효성 검증
        checkDuplicatedEmail(userSignupCommand.email)
        checkExistsPosition(userSignupCommand.positionId)
        checkExistsSubscription(SubscriptionsType.FREE.name)
        checkExistsGrade(GradesType.ENTRY.name)

        // 회원유형에 맞는 포트 조회 - 일반회원
        val userPositionCreatePort = getCreateUserPositionPortByUserType(
            UserType.NORMAL, PositionType.NORMAL
        )
        val userSubscriptionCreatePort = getCreateUserSubscriptionPort(
            UserType.NORMAL, SubscriptionsType.FREE
        )
        val userGraderCreatePort = getCreateUserGradePort(UserType.NORMAL, GradesType.ENTRY)

        // 회원등록 처리 비즈니스 로직
        val sodamUser = createNormalUserPort.createUser(userSignupCommand)
        userPositionCreatePort.createByPositionId(userId = sodamUser.userId, positionId = userSignupCommand.positionId)
        userSubscriptionCreatePort.createSubscription(userId = sodamUser.userId, subscriptionId = userSignupCommand.subscriptionId)
        userGraderCreatePort.createGrade(userId = sodamUser.userId, gradeId = userSignupCommand.gradeId)
        return sodamUser.toSignupResponse()
    }
}

```

- 회원 서비스 오브젝트에서는 비즈니스 로직만 관리하여 가독성 높고 변경에 유리한 코드를 작성할 수 있게됨